

OSWG

Oddly Specific Wordlist Generator

Version 0.7.0

Developer Guide

Repository: <https://github.com/dmarakom6/oswg>

Version: 0.7.0

License: MIT

Homebrew tap: [dmarakom6/homebrew-oswg](#)

Docker image: [ghcr.io/dmarakom6/oswg](#)

Generated: September 24, 2026

Official documentation - generated from the repository.

1 Architecture

OSWG is a single Python package with an embedded web front end. A **FastAPI** application in `src/oswg/` provides the REST API, serves the pre-built **SvelteKit** dashboard from `src/oswg/static`, and runs every job as an async task with **WebSocket** progress updates. Scraping uses `httpx` and `BeautifulSoup`; **JavaScript rendering**, interactive login, and session capture are delegated to **Playwright**. Jobs and saved templates are stored in a **SQLite** database accessed through `aiosqlite`.

Component	Role
FastAPI backend (<code>src/oswg/</code>)	REST endpoints, job orchestration, WebSocket updates.
SvelteKit UI (<code>ui/</code>)	Dashboard; the production build is copied into <code>src/oswg/static</code> and served by the backend.
SQLite store	Jobs and templates persisted via <code>aiosqlite</code> .
WebSocket	Real-time job progress pushed to the dashboard.
Playwright	JS rendering, interactive login, session capture.

1.1 Repository layout

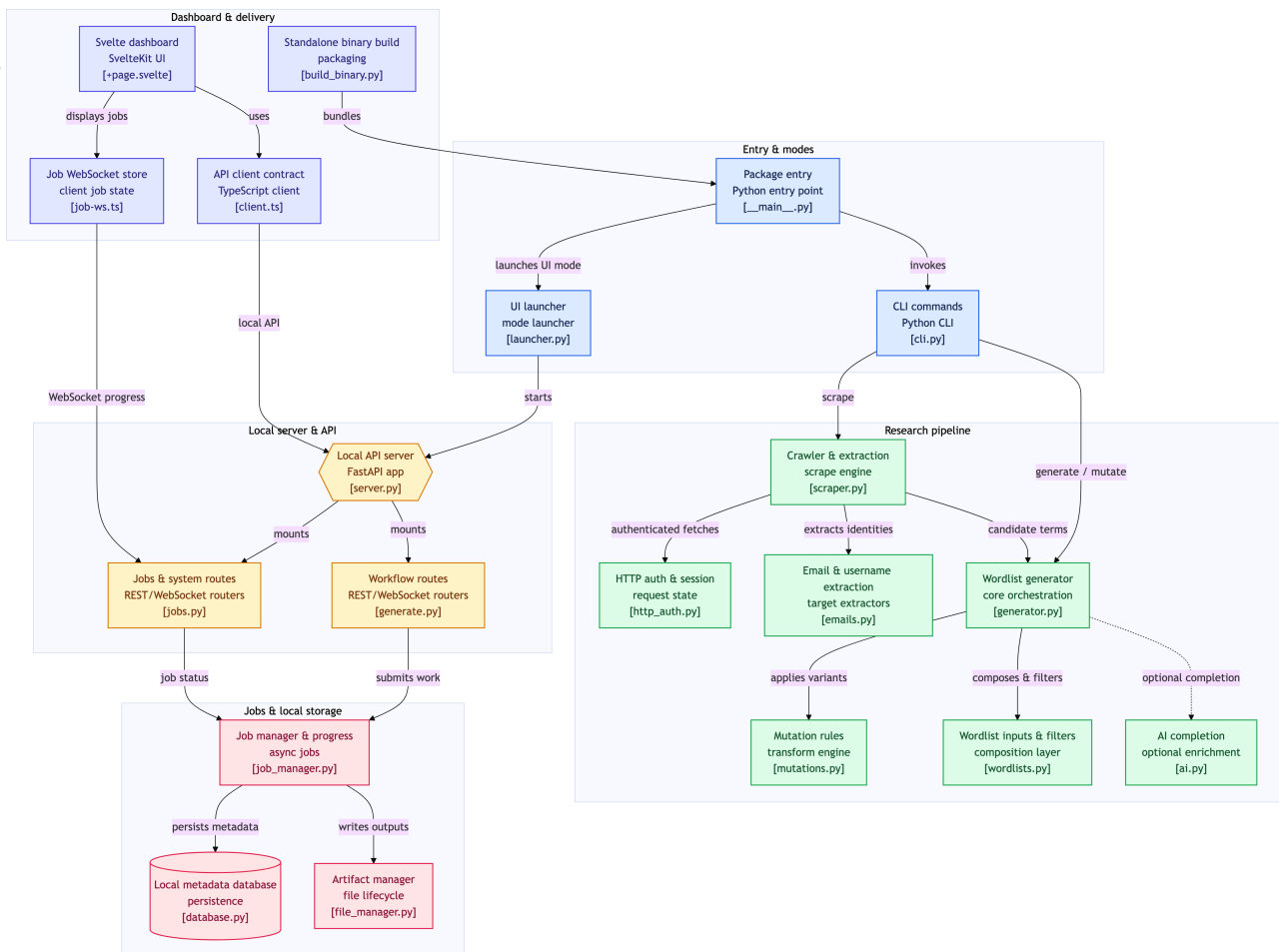
```

oswg/
|-- src/oswg/
|   |-- __init__.py      package version
|   |-- cli.py           typer CLI: generate, scrape, mutate, test, ...
|   |-- launcher.py      uvicorn launcher for `oswg ui`
|   |-- database.py      SQLite job/template store
|   |-- models.py        Pydantic request/response models
|   |-- config.py        settings (XDG data dir, retention)
|   |-- core/
|   |   |-- scraper.py    crawler: httpx, robots, rate limit, JS
|   |   |-- generator.py  wordlist generation pipeline
|   |   |-- mutations.py  mutation engine
|   |   |-- presets.py    quick/standard/aggressive/extreme
|   |   |-- rulegen.py    jtr/hashcat rule emission
|   |   |-- ai.py         Ollama/OpenAI completions
|   |   |-- test_runner.py external cracker drivers
|   |   |-- session.py    Playwright login and sessions
|   |   |-- emails.py, usernames.py, wordlists.py, ...
|   |-- routers/         FastAPI routers (generate, scrape, jobs, ...)
|   |-- services/        job manager, template store
|   |-- static/          SvelteKit build output
|-- ui/                  SvelteKit frontend source
|-- tests/               pytest unit + e2e suites
|-- build_binary.py      PyInstaller build helper
|-- oswg.spec            PyInstaller spec
|-- Makefile             build/dev/lint/test/hooks targets
|-- pyproject.toml       packaging, ruff, pytest configuration
`-- .github/workflows/    ci.yml, release.yml, release-binaries.yml

```

1.2 Architecture diagram

The diagram below maps every module to its role: entry points and CLI commands, the local FastAPI server and its routers, the research pipeline (crawler, extractors, generator, mutations, AI), the job/storage layer, and the dashboard and delivery path.



Source: docs/architecture.mmd - regenerate with python3 docs/render_architecture.py

2 Development setup

Create a virtual environment, install the package in editable mode with the dev and js extras, and install the UI dependencies:

```
python -m venv venv
source venv/bin/activate
pip install -e '.[dev,js]'
cd ui
npm ci
cd ..
```

`make build` builds the SvelteKit front end and copies `ui/build` into `src/oswg/static`, which is what the backend serves. `make dev` runs the build and starts the local server:

```
make build # npm run build in ui/, then copy static assets
make dev   # make build + python -m oswg ui
```

`make build` is required before running the app or the E2E suite - the backend serves the dashboard from `src/oswg/static`, so an empty static directory means a blank UI.

3 Testing

The project uses `ruff` and `svelte-check` for linting, plus `pytest` for a unit suite and a Playwright E2E suite.

Command

What it runs

Command	What it runs
make lint	ruff check src/ tests/ and npx svelte-check in ui/.
pytest -q	The 125 unit tests; E2E tests are deselected by default (pytest addopts -m 'not e2e').
pytest -m e2e	The Playwright E2E suite in tests/e2e/, which boots the app plus a fixture site.
make hooks	Enable the pre-push git hook (git config core.hooksPath .githubhooks).

The E2E suite requires `make build` first (the backend serves the built UI) and Playwright Chromium installed (`playwright install chromium`). The pre-push hook (`.githubhooks/pre-push`) runs ruff, the unit tests, and svelte-check before every push.

4 CI/CD

Three GitHub Actions workflows automate testing, releasing, and publishing. `ci.yml` runs 3 jobs: backend (ruff + pytest), UI (svelte-check + build), and E2E (Playwright).

Workflow	Trigger	Jobs / outputs
ci.yml	Every push and pull request	backend (ruff + pytest unit), UI (svelte-check + build), E2E (Playwright).
release.yml	Push to master	Test job, then python-semantic-release versions the package and tags vX.Y.Z.
release-binaries.yml	v* tags (or workflow_dispatch)	Binaries, PyPI, .deb, Homebrew bump, Docker image, GitHub Release.

`release-binaries.yml` builds PyInstaller binaries for `linux-x86_64`, `macos-x86_64`, and `macos-arm64`; publishes the wheel and sdist to PyPI; builds a `.deb` and attaches it to the release; bumps the Homebrew formula in the `dmarakom6/homebrew-oswg` tap; and builds and pushes the multi-arch (`linux/amd64` and `linux/arm64`) Docker image to `ghcr.io/dmarakom6/oswg`.

5 Release flow

Releases are fully automated once a version is chosen:

- Commit the version bump with a conventional commit: `chore(release): X.Y.Z`.
- Push to master. `release.yml` runs its test job, then semantic-release creates the vX.Y.Z tag.
- Pushing the vX.Y.Z tag triggers `release-binaries.yml`, which produces the binaries, the `.deb`, the PyPI package, the Homebrew bump, and the Docker image.
- The GitHub Release is created with the binaries and the `.deb` attached automatically.

Step	Trigger	Workflow	Outputs
Version commit	<code>chore(release): X.Y.Z</code> on master	<code>release.yml</code>	vX.Y.Z tag
Tag build	vX.Y.Z tag pushed	<code>release-binaries.yml</code>	binaries, .deb, PyPI, Homebrew, Docker

The current documented version is 0.7.0. Version bumps flow through the semantic-release pipeline - never edit `pyproject.toml` or `src/oswg/__init__.py` by hand for a release.